

Тема вежбе: Показивачи, низови и динамичка меморија

ПОКАЗИВАЧИ И НИЗОВИ

Показивачи

Показивач је променљива чија је вредност меморијска адреса другог податка. Показивач не садржи сам податак, већ информацију где се податак налази у меморији. Показивачи се разликују према типу податка на који показују. Основне врсте су:

- показивачи на променљиве (податке)
- показивачи на функције

Показивач се декларише на следећи начин:

```
data_type *ptr_name;
```

- ***data_type*** – тип податка на који показивач показује, не и тип показивача (тип показивача је: показивач на *data_type*,
- ***ptr_name*** – име показивача,
- ***** – у декларацији означава показивач, док у изразу врши дереференцирање.

Пример показивача на **целобројни тип** са именом **ptr**:

```
int *ptr;           или           int* ptr;
```

Показивач на неки тип може узети само адресу тог истог типа, у супротном компајлер пријављује упозорење или грешку. Једини изузетак је показивач на ***void*** тип.

```
int var;
float* fptr;
void* vptr;

fptr = &var;          /* compiler warning or error */
vptr = &var;          /* OK */
```

Додељивање вредности показивачу:

```
int *ptr;
int value = 5;
ptr = &value;
```

Дереференцирање показивача:

Дереференцирање је операција приступа вредности података на који показивач показује, односно читање или измена садржаја меморијске локације чију адресу показивач чува. Дереференцирање се врши употребом оператора *.

Оператор * има два значења:

- У декларацији означава ПОКАЗИВАЧ,
- У изразу врши дереференцирање.

Читање вредности меморијске локације на коју показивач показује

```
int a = *ptr;                /* a = value */
```

Упис у меморијску локацију на коју показивач показује

```
*ptr = 7;                    /* value = 7 */
```

НАПОМЕНА: тип и величина података који ће бити прочитани/уписани у меморију одређени су типом показивача.

Const квалификатор

Квалификатор **const** се користи за означавање података чија се вредност не сме мењати након иницијализације. Његова основна улога је спречавање ненамерних измена података и повећање безбедности програма.

Када се квалификатор **const** примени на показивач разликујемо три основна случаја:

- Показивач на константан **податак**,
- Константан **показивач**,
- Константан **показивач** на константан **податак** (оба истовремено).

Шта је константно?	Декларација	Примена
Податак	const int* p	Само читање података (нпр. испис низа)
Показивач	int* const p	Ако показивач мора да показује на исти објекат (интерни бафер)
Оба	const int* const p	Приступ конфигурационим подацима

Показивач на константан податак:

Показивач се може мењати (адреса) али не и оно на шта он показује (вредност).
Кључна реч *const* мора бити лево од ‘*’

<code>const int* ptr;</code>	или	<code>int const* ptr;</code>
------------------------------	-----	------------------------------

Пример коришћења:

<pre>int a = 3; int b = 5; const int* ptr; ptr = &a; ptr = &b; /* OK */ *ptr = 7; /* ERROR */</pre>
--

<code>void print(const int* arr);</code> // гарантује да фја неће изменити елементе

Експлицитна конверзија мора бити коришћена када вредност оваквог показивача желимо да доделимо обичном, неконстантном, показивачу

<pre>int* ptr; int const* cptr; ptr = cptr; /* compiler warning or error */ ptr = (int*) cptr; /* OK */</pre>
--

Константан показивач:

Може се мењати оно на шта показивач показује (вредност), али не и сам показивач (адреса). Кључна реч *const* мора бити десно од ‘*’

<code>int* const ptr_variable;</code>

Пример коришћења:

<pre>int a = 3; int b = 5; int* const ptr; ptr = &a; ptr = b; /* ERROR */ *ptr = 7; /* OK */</pre>
--

Константан показивач на константан податак:

Дупло константни показивач – није могућа његова измена као ни измена онога на шта он показује, тј. ни вредност ни адреса.

<code>const int* const ptr;</code>	или	<code>int const* const ptr;</code>
------------------------------------	-----	------------------------------------

Пример коришћења:

<pre>int a = 3; int b = 5; int const* const ptr = &a; ptr = b; /* ERROR */ *ptr = 7; /* ERROR */</pre>

Sizeof оператор

Унарни оператор који израчунава величину типа и изражава је у **бајтима**. Може се користити **над променљивом или над типом**. У случају да се користи над променљивом, повратна вредност је величина те променљиве тј. величина типа ког је та променљива. Када ради над типом враћа његову величину.

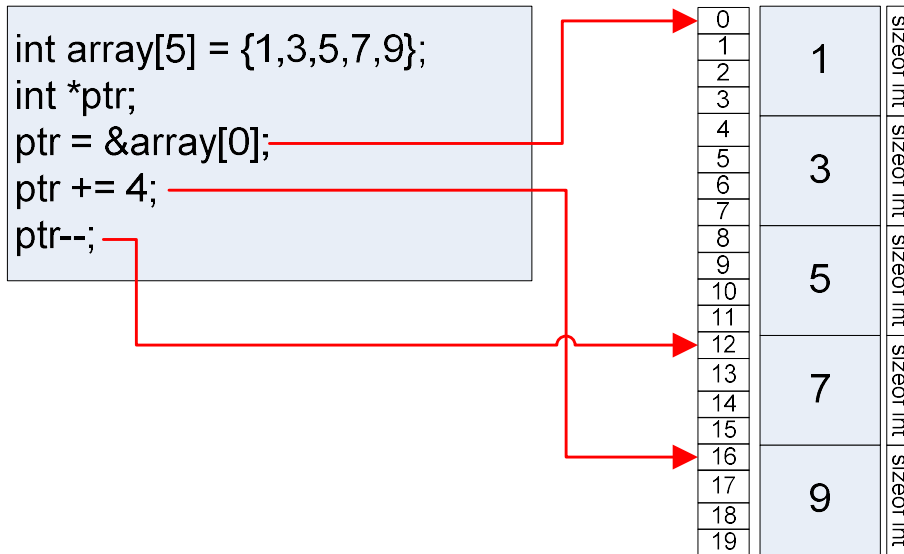
По дефиницији: `sizeof(char) = 1`

<pre>int* ptr; int array[3] = {1, 2, 3}; int s; s = sizeof(int); printf("%d\n", s); /* prints 4 */ s = sizeof(ptr); printf("%d\n", s); /* prints 4 */ s = sizeof(array); printf("%d\n", s); /* prints 12 (3*sizeof(int)) */</pre>
--

Показивачи и операције над њима

Операције сабирања и одузимања показивача и целог броја раде на следећи начин:

```
data_type* ptr;  
ptr ± n <=> ptr ± n * sizeof (data_type)
```



НАПОМЕНА: Померање показивача се врши у јединицама величине типа, а не у бајтовима. Исто важи и за унарне операторе ++ и --.

Одузимање два показивача је могуће само ако су показивачи **истог типа** и ако показивачи показују на адресе унутар истог дела меморије, у супротном резултат ће бити недефинисан. Сабирање два показивача није дозвољено по стандарду, те ће компајлер у том случају пријавити грешку. Такође, поређење има смисла само ако показивачи указују на исто парче меморије.

```
int array[5] = {1,3,5,7,9};  
int* ptr1;  
int* ptr2;  
ptr1 = &array[1];  
ptr2 = &array[4];  
printf("%d\n", (ptr2-ptr1));          /* prints 3 */  
-----  
int array1[5] = {1,3,5,7,9};  
int array2[3] = {2,4,6};  
int* ptr1;  
int* ptr2;  
ptr1 = &array1[0];  
ptr2 = &array2[2];  
printf("%d\n", (ptr2-ptr1));          /* undefined */  
int sum = ptr1 + ptr2;                /* ERROR */
```

Оператор []

По дефиницији: $A[B] \Leftrightarrow *(A + B)$

где $A+B$ мора бити показивачког типа јер оператор $*$ ради само са показивачем. Дакле или A мора бити показивач, а B целобројног типа, или обрнуто:

```
data_type* A;  
int B;  
A[B]  $\Leftrightarrow *(A + B) \Leftrightarrow *(A + B * \text{sizeof}(data\_type))$   
B[A]  $\Leftrightarrow *(B + A) \Leftrightarrow *(A + B * \text{sizeof}(data\_type))$ 
```

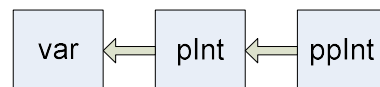
Пример:

```
float* p;  
float x;  
/* neka je p 1000, tj. p pokazuje na adresu 1000 */  
  
x = *p;      // x je float vrednost sa adrese 1000  
x = p[0];    // x je float vrednost sa adrese 1000  
x = p[4];    // x je float vrednost sa adrese (1000 + 4*sizeof(float))  
x = 4[p];    // x je float vrednost sa adrese (1000 + 4*sizeof(float))
```

Вишеструки показивачи

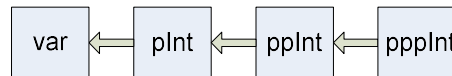
Показивач може показивати на било који тип, па тако и на тип показивача. Вишеструки показивачи се користе када је потребно мењати сам показивач у функцији или представити динамичке вишедимензионалне структуре:

```
int var;  
int* pInt = &var;  
int** ppInt = &pInt;
```



и то тако унедоглед:

```
int var;  
int* pInt = &var;  
int** ppInt = &pInt;  
int*** pppInt = &ppInt;
```



Вишеструки показивачи су потребни у два сценарија:

- када је у програму неопходно проследити показивач по референци,
- у случају рада са вишедимензионалним низовима.

Прослеђивање показивача по референци:

```
int g_var;

void bar(int** p)
{
    *p = &g_var;    // change pointer value
    **p = 39;        // change value of variable to which
                    // pointer points to
}

void foo()
{
    int var;
    int* ptr = &var;
    bar (&ptr);
    . . .
}
```

Вишедимензионални низови:

```
void bar(int** mat)
{
    mat[0][0] = 1;
}

void foo()
{
    int** matrix;
    matrix = new int* [2];
    matrix[0] = new int[2];
    matrix[1] = new int[2];
    bar(matrix);
}
```

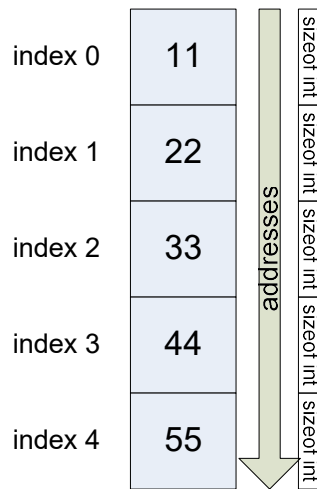
Низови

Низ у програмском језику Це представља блок меморије који се тумачи као да се у њему, један за другим, налазе елементи исте величине. Елементи низа су континуалне меморијске локације. Декларација низа:

```
element_type array_name[dimension] = {declaration_list};
```

Пристап елементу низа обавља се помоћу оператора []:

```
int array[5] = {11, 22, 33, 44, 55};  
printf("element with index 3: %d\n", array[3]); // prints 44
```



Као и код регуларних променљивих, ако нема експлицитне иницијализације, низови статичке трајности постављају се на 0 (сви елементи), а аутоматске остају недефинисане:

int array[5];	local	global
	NDF NDF NDF NDF NDF	0 0 0 0 0

Експлицитна иницијализација се реализује набрајањем елемената у витичастим заградама. Листа не сме имати више елемената од димензије низа. Ако има мање, преостали елементи се попуњавају нулама.

int array[5] = {1, 3, 5};	1	3	5	0	0

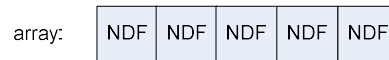
У случају да је потребно иницијализовати само неке елементе низа:

```
int array[100] = {[13] = 5, [77] = 6};
```

Однос низова и показивача

Иако је већ речено да су низови и показивачи врло слични, ипак постоје разлике. Када се дефинише низ, заузима се меморија за све његове елементе. Када су у питању показивачи то није случај, јер се меморија заузима само за тај показивач.

```
char array[5];
```



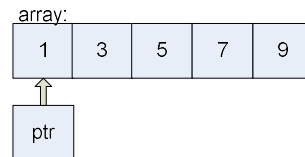
```
char* ptr;
```



Низ, односно име низа, представља његову адресу, адресу почетка низа (првог елемента). Показивач је са друге стране само променљива чија је вредност нека адреса. У том смислу се низ може посматрати као показивачка константа.

```
int array[] = {1, 3, 5, 7, 9};
```

```
int* ptr = array;
```



Поред претходно наведених разлика, разлика низова и показивача се огледа и у резултату *sizeof* операције:

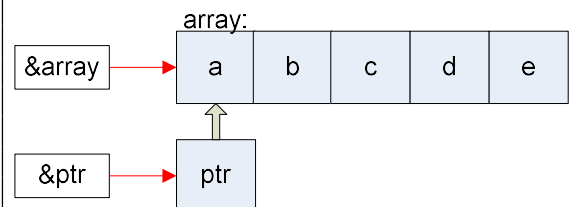
- за низ враћа број меморијских речи које су заузеле за цео низ
- за показивач враћа колико меморијских речи је заузето за адресу

```
char array[15];  
char* ptr;  
printf("%d\n", sizeof(array));      /* prints 15 */  
printf("%d\n", sizeof(ptr));       /* prints 4 */
```

Разлика у & оператору:

- За низ враћа адресу првог елемента (исто што и &array[0])
- За показивач враћа адресу показивача (на којој адреси се налази показивач као променљива)

```
char array[] = {'a', 'b', 'c', 'd', 'e'};  
char* ptr = array;  
  
if (array[0] == ptr[0])      /* equal */  
    printf("equal");  
else  
    printf("not equal");  
if (&array == &ptr)          /* not */  
    printf("equal");  
else  
    printf("not equal");
```



Додела меморије

С језик подржава два начина заузимања меморије кроз дефинисање променљивих:

- **Статичко заузимање** – које се примењује за све глобалне и статички дефинисане променљиве. Свака променљива одређује један блок меморије, непроменљиве величине. Меморија се заузима једном (приликом покретања програма) и никад се не ослобађа.
- **Аутоматско заузимање** – које се примењује за аутоматске променљиве тј. за аргументе функција или локално дефинисане променљиве. Меморијски простор за овакве променљиве се заузима сваки пут кад се у току извршавања програма уђе у програмски блок који садржи дефиницију променљиве, а ослобађа се на крају тог програмског блока.

Заједничко за статичко и аутоматско заузимање меморије је да величина коју је потребно заузети мора бити константна вредност:

```
static int var = 5;  
int array1[10];  
int array2[var]; /* compiler error */
```

Трећи начин заузимања меморије – динамичко заузимање није подржано преко Це променљивих, али је доступно преко стандардних библиотека (*stdlib*) и преко C++ оператора.

- **Динамичко заузимање** – техника која се примењује у случајевима када информације о величини и трајању променљивих нису доступне пре почетка извршавања програма. Представља акцију где програмер експлицитно затражи заузимање меморије. С обзиром да заузимањем управља програмер, дужан је и да заузету меморију на исправан начин ослободи. У супротном може доћи до грешака које је тешко уочити и отклонити (цурење меморије). Цурење меморије настаје када програм изгуби адресу заузете меморије без њеног ослобађања.

У С језику функције стандардне библиотеке (*stdlib*) за динамичко заузимање и ослобађање меморије су:

- `calloc()` – заузимање и постављање заузете меморије на вредност 0
- `malloc()` – заузимање меморије
- `realloc()` – реалокација претходно заузете меморијске зоне
- `free()` – ослобађање заузете меморије

У C++ језику на располагању су оператори:

- `new` – заузимање меморије
- `new[]` – заузимање меморије за низ елемената
 - повратна вредност `new` оператора је показивач на почетак заузете меморијске зоне
- `delete` – ослобађање меморије
- `delete[]` – ослобађање меморије за заузет низ елемената
 - `delete` оператори се користе над показивачем који показује на почетак меморијске зоне коју треба ослободити

НАПОМЕНА: Да би се избегле грешке у пару увек треба користити:

- `calloc/malloc – free`
- `new – delete`
- `new[] – delete[]`

Од интереса за овај предмет ће бити C++ оператори, а њихова примена описана је у примерима који следе.

Пример `new` и `delete` оператора:

```
int* ptr = NULL;
ptr = new int;           // allocates memory for one integer and
                        // returns a pointer to that object
*ptr = 10;              // sets the object value to 10 */
delete ptr;             // deletes the object (frees the memory)
```

```
int* ptr = new int(5);
// allocates memory for one integer with value 5

delete ptr;             // frees the memory
```

Пример: `new[]` и `delete[]` оператора:

```
int arraySize = 0;
int* array;
cout << "Please enter desired array size: ";
cin >> arraySize;
array = new int[arraySize];
. . . // do something with array
delete[] array;
```

```
int** matrix;
int rows = 2;
int cols = 2;
matrix = new int* [rows];
// allocates memory for an array of pointers and returns
// a pointer to the allocated array of pointers
// note: return value is pointer to a pointer

for (int i = 0; i < rows; i++)
    matrix[i] = new int[cols];
// allocating memory for each pointer from the array of
// pointers

for (int i = 0; i < rows; i++)
    delete[] matrix[i];
// freeing memory of each array

delete[] matrix;
// freeing memory for the array of pointers
```

Вишедимензионални низови

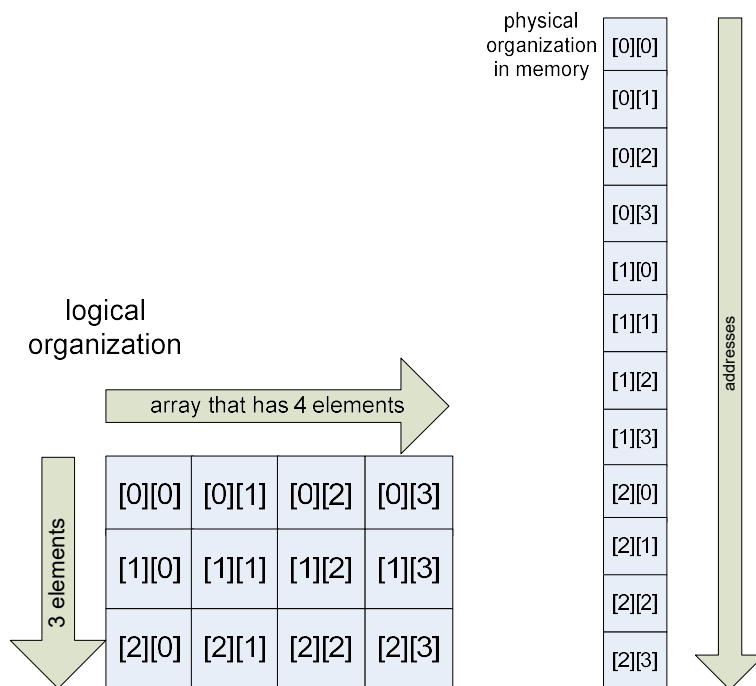
У Це-у вишедимензионални низови су у ствари низови низова. Ако говоримо о дводимензионалном низу то је у суштини низ елемената од којих је сваки елемент тог низа, опет низ. Постоји разлика у начину заузимања меморије ког дводимензионалних низова и то само у физичкој организацији, што је објашњено у примерима у наставку:

Пример статички заузетог дводимензионалног низа:

```
int matrix[3][4];
```

- то је низ који има 3 елемента
- Питање: Ког су типа та 3 елемента?
- Одговор: Елементи тог низа су типа низ од 4 целобројне вредности

Логичка и физичка организацији дата је на слици:



Еквивалентна дефиниција горенаведеног низа је:

```
typedef int array4[4];
array4 matrix[3];           // equals to int matrix[3][4];
array4 y;                   // equal to int y[4];
int i;
int j;

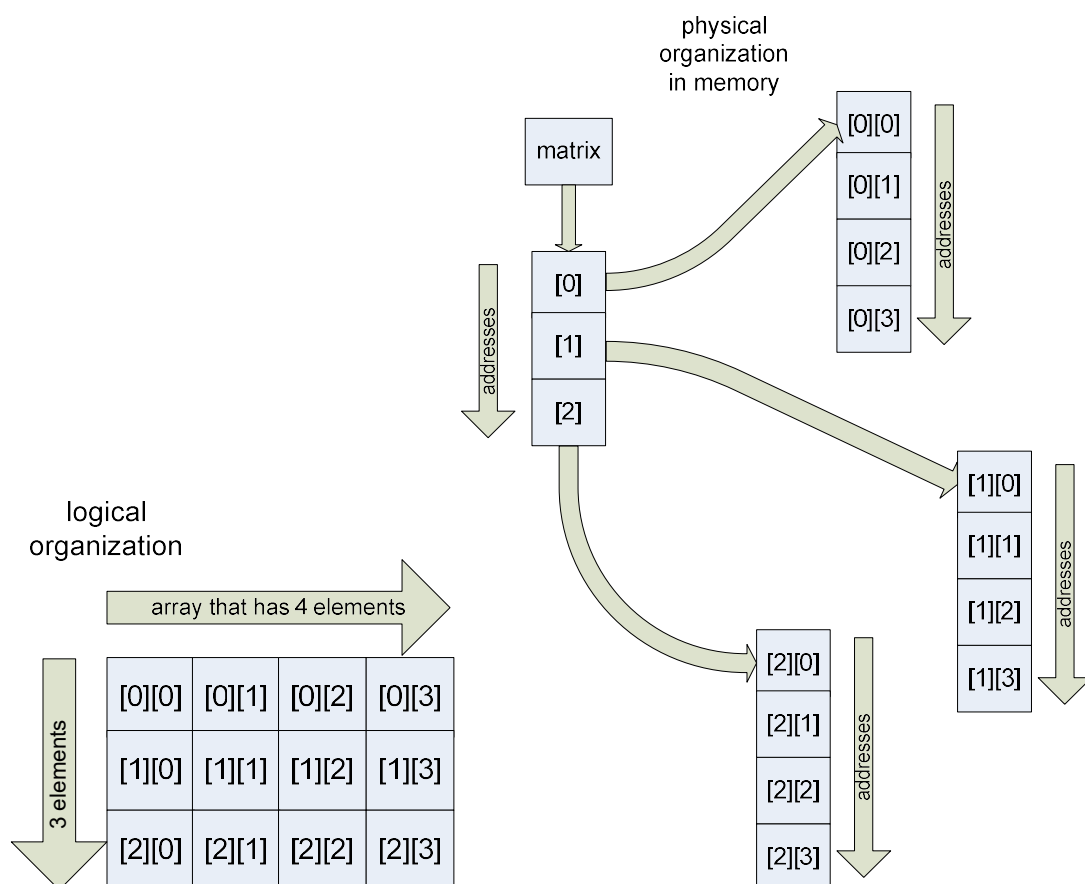
y[i];                       // equal to *(y + i*sizeof(int))

matrix[i][j];
// equals to
// (matrix[i])[j]
// (*(matrix + i*sizeof(array4)))[j] // array4 T
// T[j]
// *(T + j*sizeof(int))
// *((*(matrix + i*sizeof(array4))) + j*sizeof(int))
// sizeof(array4) = 4*sizeof(int)
// *((*(matrix + i*4*sizeof(int)) + j*sizeof(int))
```

Пример динамички заузетог дводимензионалног низа:

```
int** matrix;  
matrix = new int*[3];  
matrix[0] = new int[4];  
matrix[1] = new int[4];  
matrix[2] = new int[4];  
  
matrix[i][j];  
// equals to  
// (matrix[i])[j]  
// (*(matrix + i*sizeof(int*)))[j]  
// T[j]  
// *(T + j*sizeof(int))  
// *(* (matrix + i*sizeof(int*)) + j*sizeof(int))  
// *(* (matrix + i*4) + j*4)
```

Логичка и физичка организација дата је на слици:



Прослеђивање низова функцији

Низови се функцијама не могу проследити по вредности. Прослеђивање се увек обавља на тај начин што се проследи **адреса првог елемента**. Последица је да су наведене декларације практично исте:

```
int func(int arr[10]);  
int func(int arr[]);  
int func(int* arr);
```

Број елемената наведен у угластим заградама се игнорише. За вишедимензионалне низове, све димензије се морају навести осим прве.

```
int func(int mat[][7]);  
int func(int (*)[7]);  
int func(int (*mat)[7]);
```

Уколико смо користили динамичко заузимање меморије за дводимензионални низ, тада се ради о показивачу на показивач па декларација и позив функције изгледају овако:

```
void initMatrix(int** mat, int rowNumber, int colNumber)  
{  
    for (int i = 0; i < rowNumber; i++)  
        for (int j = 0; j < colNumber; j++)  
            mat[i][j] = 0;  
}  
  
void foo()  
{  
    int** matrix;  
    int rows = 2;  
    int cols = 2;  
    matrix = new int* [rows];  
  
    for (int i = 0; i < rows; i++)  
        matrix[i] = new int[cols];  
  
    initMatrix(matrix, rows, cols);  
  
    for (int i = 0; i < rows; i++)  
        delete[] matrix[i];  
  
    delete[] matrix;  
}
```

ЗАДАТАК

I. Написати програм за рад са матрицом од $n \times m$ целобројних елемената који садржи три класе у истоименим датотекама типа `.h` и `.cpp`:

- ***Matrix1D*** – матрица као једнодимензионални низ
- ***Matrix2D*** – матрица као низ показивача на низове (редови матрица),
- ***MatrixVec*** – матрица као вектор вектора (користити класу `std::vector`).

За сваку од три класе потребно је:

- У заглављу класе додати приватно поље које представља матрицу.
- Имплементирати иницијализацију матрице у оквиру конструктора. Конструктор прима три параметра: број редова (***rows***), број колона (***cols***) и параметар за одређивање максималне вредности елемента матрице (***range***). Попуњавање елемената матрице свести на доделу случајних вредности које ће бити мање или једнаке од датог параметра *range*. Случајне вредности генерисати употребом функције:
int rand ();
- Имплементирати методу за испис матрице на екран:
void print();
 - Испис форматирати тако да сваки ред матрице буде исписан у новом реду, а елементи унутар једног реда одвојени са `"\t"` карактером.
- Имплементирати методу која рачуна број непарних елемената у свакој од колона матрице и исписује на екран:
void process();
- У функцији ***main*** направити објекат за сваку од три класе и позвати редом методе *print* па *process* над њима.
- Задавање димензија матрице n и m реализовати уз помоћ аргумената командне линије

ДОДАТНИ ЗАДАТАК

У сваку класу додати функцију за ротирање матрице за 90 степени у смеру казаљке на сату. Ограничити се само на квадратне матрице.